

Appendix E: Commented Arduino code

```

/*****
Overall arduino code for the robot competition. The idea and strat is that we
just intialize everything, deploy the catching net, then start out movement, whereas
we just go forward until we hit a wall, then depending on the QTI sensor readings
we will either turn around, make a right, or make a left. Very simple
*****/

#include <Arduino.h>
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#define SERVO_PIN PB3

/* ----- Prototypes ----- */
void    pwm_setup(void);
void    initQTI(void);
inline  bool qti1IsBlack(void);
inline  bool qti2IsBlack(void);
void    driveLeft(void);
void    stopAll(void);

/* ===== */
int main(void)
{
    init();           // Arduino core start-up
    Serial.begin(9600);
    //basically just starting everything up
    _delay_ms(2000);
    pwm_setup();
    initQTI();
    //all the function above will be explained below
    DDRB |= (1 << SERVO_PIN);
    //just using piecwise operations so we can start the unrolling process. I.e this
means make the pin connected
    // to our servo an output pin so we can use it and run it high.
    unrollFor5s();
    Serial.print("unrolling");

    while (1)
        //while statement that will run the loop forever

```

```

{

    if ((PINB & (1 << PB4)) && (PIND & (1 << PD3))) {
        turnaround();
        Serial.println(F("QTI's BLACK → turnaround"));
    }
    //this first if statement basically says if both QTI's read black,
    // then turn around

    else if (PINB & (1 << PB4))
    {
        Serial.println(F("QTI-2 BLACK → right TURN"));
        driveRight();
    }
    //this second if statement basically says if the left QTI is reading black
    // then turn right

    else if (PIND & (1 << PD3))    // right-side sensor → turn right
    {
        Serial.println(F("QTI-1 BLACK → left TURN"));
        driveleft();
    }
    //this last if statement basically says if the right QTI is reading black
    // then turn left

    }
    else                // no black under either sensor
    {
        driveForward();
    }
    //the final command in the lopp is that we aren't reading black then keep
moving forward

    _delay_ms(50);
    //this part is just a little delay before the loop is ran again
}
}

//functions

void initQTI(void)
{

```

```

/* Provide 5 V to both sensors via two spare I/O pins */
DDRB |= (1 << PB5);   PORTB |= (1 << PB5);   // D13 HIGH
DDRD |= (1 << PD7);   PORTD |= (1 << PD7);   // D7  HIGH

/* Sensor outputs as inputs, no pull-ups */
DDRB &= ~(1 << PB4);  PORTB &= ~(1 << PB4);   // D12
DDRD &= ~(1 << PD3);  PORTD &= ~(1 << PD3);   // D3
}

//This first function is just to initialize the QTI sensors
// Just setting the appropriate pins to input/output

/* ----- PWM and motor channels ----- */
void pwm_setup(void)
{
    /* ----- Timer-0 (8-bit) → D5 / D6 ----- */
    DDRD |= (1 << PD5) | (1 << PD6);           // outputs
    TCCR0A = (1 << COM0A1) | (1 << COM0B1) |
              (1 << WGM01) | (1 << WGM00);      // fast-PWM
    TCCR0B = (1 << CS01);                       // prescale /8

    /* ----- Timer-1 (16-bit) → D9 / D10 ----- */
    DDRB |= (1 << PB1) | (1 << PB2);           // outputs
    ICR1  = 0xFFFF;                           // TOP
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) |
              (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) |
              (1 << CS11);                     // prescale /8
}

//this next part sets up our pwm duty cycle motor control stuff.
//basically when we first started with motor control we realized that one
// of our motors spins after then the other, so we had to implement a duty
//cycle on both of them to remedy it.

/* ----- Motor routines ----- */
/* duty values picked for ≈85-95 % PWM */
static const uint8_t  duty8  = 220;           // 8-bit (0-255)
static const uint16_t duty16 = 62000;        // 16-bit (0-65535)

/* ----- NEW helper: one 20 ms servo frame ----- */
static inline void servoPulse(uint16_t high_us)

```

```

{
    PORTB |= (1 << SERVO_PIN);          // HIGH
    _delay_us(high_us);                  // e.g. 500-2500 µs
    PORTB &= ~(1 << SERVO_PIN);          // LOW
    _delay_ms(20 - high_us / 1000);      // finish 20 ms frame
}

//This function is just a helper function for unrolling the hobby servo
//this kind of just decides the direction in which the servo spins or it holds
// its position

/* ----- REPLACE your old unroll() with this ----- */
void unrollFor5s(void)
{
    const uint16_t pulseWidth = 1800;    // "unroll" position
    const uint16_t frames      = 1600 / 20; // ≈5 s / 20 ms

    for (uint16_t i = 0; i < frames; ++i)
        servoPulse(pulseWidth);
}

//this is just the actual function for the unrolling.
// you give it a pulse width that will determine if it holds, spins left or right,
then a frame
// which is basically the time of the unroll

int driveForward() {
    OCR0A = 238; OCR0B = 0;
    OCR1A = 60000; OCR1B = 0;
    Serial.println("forward");
    return 0;
}

//just drives the pwm in a certain way such that robot moves forward

int driveleft() {
    OCR0A = 219; OCR0B = 0;
    OCR1A = 0; OCR1B = 62000;
    _delay_ms(1000);
    Serial.println("left");
    return 0;
}

//just drives the pwm in a certain way such that robot turns left

int turnaround(){

```

```
OCR0A = 219; OCR0B = 0;
OCR1A = 0;   OCR1B = 62000;
_delay_ms(2000);
Serial.println("around");
}
//just drives the pwm in a certain way such that robot turns around

int driveRight() {
    OCR0A = 0;   OCR0B = 219;
    OCR1A = 64000; OCR1B = 0;
    _delay_ms(800);
    Serial.println("right");
    return 0;
}
//just drives the pwm in a certain way such that robot turns right

void stopAll(void)
{
    OCR0A = OCR0B = OCR1A = OCR1B = 0;
}
//just a stop to all drive motor commands
```